

Umsetzungsplan – Projekt 10K

Multiplayer-Würfelspiel | Android-App + PHP/MySQL-Backend

Neuimplementierung von Grund auf | Gesamtumfang: 54 Stunden

Phasenuebersicht

Phase	Bezeichnung	Stunden
Phase 1	Analyse & Planung	4 h
Phase 2	Datenbankentwicklung	4 h
Phase 3	Backend: Grundstruktur & Konfiguration	3 h
Phase 4	Backend: Spiellogik & API-Endpunkte	9 h
Phase 5	Android: Projektaufbau & Grundstruktur	8 h
Phase 6	Android: Lobby	6 h
Phase 7	Android: Spielfeld & Spielmechanik	10 h
Phase 8	Tests & Deployment	6 h
Phase 9	Dokumentation	4 h
Gesamt		54 h

Phase 1 – Analyse & Planung (4 Stunden)

Ziel: Vollstaendiges Verstaendnis der Anforderungen, bevor eine Zeile Code geschrieben wird.

#	Was	Wie	Ergebnis
1.1	Spielregeln vollstaendig definieren	Alle Wertungen aufschreiben: 1er, 5er, Dreierpasch bis Funferpasch, Hot Dice, Einsteigsbedingung	Regelblatt als Referenz
1.2	Use Cases beschreiben	Wer macht was? Spieler erstellt Raum, tritt bei, wurfelt, wahlt, bankt, KI spielt	Use-Case-Liste
1.3	Datenbankschema entwerfen	3 Tabellen: games, players, turns – Felder, Typen, Beziehungen skizzieren	ERM-Entwurf (Papier oder Draw.io)
1.4	API-Endpunkte spezifizieren	Liste aller Endpunkte mit URL, Methode, Request-Body, Response-Feldern	API-Spezifikation
1.5	Anforderungen festhalten	Funktionale und nicht-funktionale Anforderungen, Abgrenzung	Anforderungsdokument

Phase 2 – Datenbankentwicklung (4 Stunden)

Ziel: Lauffaehige Datenbankstruktur lokal und auf dem Produktivserver.

#	Was	Wie	Ergebnis
2.1	Lokale MySQL-Datenbank anlegen	XAMPP starten, phpMyAdmin, neue DB z.B. 'zehntausend'	Leere DB vorhanden
2.2	Tabelle: zehntausend_games	SQL: id, code, status, current_turn, win_score, timestamps	Tabelle angelegt
2.3	Tabelle: zehntausend_players	SQL: id, game_id, slot, name, token, is_ai, total_score, has_entered	Tabelle angelegt
2.4	Tabelle: zehntausend_turns	SQL: id, game_id, player_id, turn_no, roll_no, dice_json, kept_json, turn_score, action	Tabelle angelegt
2.5	setup.sql erstellen	Alle CREATE TABLE-Anweisungen in eine Datei	setup.sql fertig
2.6	DB auf kronisoft.net anlegen	phpMyAdmin im Confixx-Panel, setup.sql importieren	Produktion bereit
2.7	Verbindungstest	Einfaches PHP-Skript: PDO-Verbindung pruefen, SELECT 1	Kein Fehler, Verbindung OK

Phase 3 – Backend: Grundstruktur & Konfiguration (3 Stunden)

Ziel: Projektordner, Konfiguration, Umgebungserkennung und Hilfsfunktionen bereitstellen.

#	Was	Wie	Ergebnis
3.1	Ordnerstruktur anlegen	projekte/10k/ mit Unterordner api/ lokal erstellen	Struktur vorhanden
3.2	Git-Repository initialisieren	git init, .gitignore (config.php ausschliessen), erstes Commit	Repo mit erstem Commit
3.3	config.php schreiben	DB-Konstanten, PDO-Singleton, Umgebungserkennung per HTTP_HOST	Datenbankzugriff zentral
3.4	_helpers.php schreiben	ok()/err()-Wrapper, input(), playerByToken(), gamePlayers(), genCode()	Hilfsfunktionen verfuegbar
3.5	.htaccess anlegen	Options -Indexes, php_flag display_errors off	.htaccess fertig
3.6	CORS-Header in _helpers.php	Access-Control-Allow-Origin: * fuer App-Zugriff	API von App erreichbar

Phase 4 – Backend: Spiellogik & API-Endpunkte (9 Stunden)

Ziel: Alle API-Endpunkte implementieren und die vollstaendige Spiellogik abbilden.

#	Was	Wie	Ergebnis
4.1	scoring.php – Wertungsregeln	Funktion allOptions(): alle Kombinationen berechnen, validateSelection(), hasScoringOption()	Scoring-Engine testbar
4.2	scoring.php – manuell testen	Kleines Testskript: allOptions([1,1,1,5,3]) ausgeben	Korrekte Punkte bestaetigt
4.3	api/create.php	Raum anlegen, Code generieren, Host-Spieler + KI-Spieler einfuegen	POST /api/create.php
4.4	api/join.php	Freien Slot finden, neuen Spieler einfuegen, Token zurueckgeben	POST /api/join.php
4.5	api/start.php	Status auf 'running' setzen, mind. 2 Spieler pruefen	POST /api/start.php
4.6	api/roll.php	Wurfeln, Bust-Pruefung, Wuerfel in DB speichern, Optionen zurueckgeben	POST /api/roll.php
4.7	api/keep.php	Auswahl validieren, Score addieren, Wuerfel als kept markieren	POST /api/keep.php
4.8	api/bank.php	Punkte gutschreiben, Einsteigsbedingung pruefen, Gewinnpruefung, naechster Spieler	POST /api/bank.php
4.9	api/state.php	Spielzustand, alle Spieler, letzter Wurf, Log, Gewinner zurueckgeben	GET /api/state.php
4.10	api/ai_turn.php	KI-Zug: greedy wurfeln, banken wenn > 300 Pkt oder 1000 Einstieg	POST /api/ai_turn.php
4.11	Alle Endpunkte mit curl testen	curl -X POST ... fuer jeden Endpunkt, JSON-Antworten pruefen	Backend vollstaendig validiert
4.12	Deployment auf kronisoft.net	FileZilla: alle PHP-Dateien hochladen, Berechtigungen 644/755	Backend live

Phase 5 – Android: Projektaufbau & Grundstruktur (8 Stunden)

Ziel: Android-Projekt laeuft, baut erfolgreich und kommuniziert mit dem Backend.

#	Was	Wie	Ergebnis
5.1	Neues Android-Projekt anlegen	Android Studio: Empty Activity, Kotlin, min SDK 26	Leeres Projekt baut
5.2	Git-Repository verbinden	git init oder vorhandenes Repo clonen	Versionierung aktiv
5.3	Dependencies in build.gradle	Compose BOM, Retrofit, OkHttp, Gson, Coroutines, Navigation	Gradle sync erfolgreich

#	Was	Wie	Ergebnis
5.4	Datenmodelle schreiben	Kotlin Data Classes fuer alle Request/Response-Typen, Gson-Annotationen	models/Models.kt fertig
5.5	RetrofitClient schreiben	Singleton, BASE_URL Prod/Lokal, OkHttpClient mit Logging	api/RetrofitClient.kt
5.6	ApiService-Interface schreiben	Alle 8 Endpunkte als suspend fun mit @GET/@POST	api/ApiService.kt
5.7	SessionStore schreiben	SharedPreferences: save(), getToken(), clear()	util/SessionStore.kt
5.8	App-Theme definieren	Material3 Dark Color Scheme, Farben, Schriften	ui/Theme.kt
5.9	Navigation einrichten	NavHost mit 2 Routen: lobby, game	MainActivity.kt
5.10	Ersten API-Aufruf testen	Einfacher curl-ersatz: state.php mit festem Token aufrufen, Log pruefen	Netzwerk funktioniert

Phase 6 – Android: Lobby (6 Stunden)

Ziel: Spieler koennen einen Raum erstellen, beitreten und auf den Spielstart warten.

#	Was	Wie	Ergebnis
6.1	LobbyViewModel: createGame()	Retrofit-Call, Token + Gameld in SessionStore speichern, bei KI direkt starten	Raum erstellbar
6.2	LobbyViewModel: joinGame()	Code + Name senden, Token empfangen und speichern	Beitreten funktioniert
6.3	LobbyViewModel: startGame()	Nur Host, Status-Pruefung, navigiert zu game	Spielstart ausloesbar
6.4	LobbyScreen: Erstellen-Card	Name, KI-Anzahl (FilterChips), Zielpunktzahl, Button	UI fertig
6.5	LobbyScreen: Beitreten-Card	Name, Code-Eingabe (Grossbuchstaben), Button	UI fertig
6.6	LobbyScreen: Warteraum	Raumcode gross anzeigen, Spielerliste, Start-Button fuer Host	Warteraum fertig
6.7	Polling im Warteraum	Alle 2s state.php abfragen, bei status=running zu game navigieren	Gast startet automatisch
6.8	Fehlerzustaende anzeigen	ErrorBanner-Composable, Fehlermeldungen aus API anzeigen	Fehlerbehandlung fertig

Phase 7 – Android: Spielfeld & Spielmechanik (10 Stunden)

Ziel: Vollstaendig spielbares Spielfeld mit allen Interaktionen und Rueckmeldungen.

#	Was	Wie	Ergebnis
7.1	DieView-Composable	Box mit Wurfelzahl, 3 Zustände: aktiv, ausgewaehlt, behalten, Animation	Einzelner Wurfel darstellbar
7.2	GameViewModel: Polling	Coroutine alle 2s, StateFlow fuer gameState, stopPolling() bei finished	Zustand wird aktualisiert
7.3	GameViewModel: roll()	API-Call, Ergebnis in activeDice/keptDice/turnScore, Bust-Handling	Wurfeln funktioniert
7.4	GameViewModel: toggleDie()	Index in selectedIdx toggeln, nur im Phase ROLLED erlaubt	Wuerfel waehlen funktioniert
7.5	GameViewModel: keep()	API-Call mit selected-Liste, keptDice + remaining aktualisieren	Behalten funktioniert
7.6	GameViewModel: bank()	API-Call, turnUI zuruecksetzen, naechster Spieler via Poll	Banken funktioniert
7.7	GameScreen: Wuerfelfeld	DieView-Zeile: behaltene grau, aktive klickbar, Platzhalter wenn leer	Wuerfel sichtbar
7.8	GameScreen: Aktionsbuttons	Wurfeln / Behalten / Banken mit korrekten enabled-Bedingungen	Buttons reagieren korrekt
7.9	GameScreen: Scoreboard	Alle Spieler, Fortschrittsbalken, aktiver Spieler hervorgehoben	Punkte sichtbar
7.10	GameScreen: Spiellog	LazyColumn, letzte 10 Aktionen, farbcodiert nach Typ	Log sichtbar
7.11	Bust-Feedback	Rotes Banner + Animation auf den Wurfeln	Bust klar erkennbar
7.12	Hot-Dice-Feedback	Hinweis wenn alle 5 Wurfel behalten: neue Runde mit 5 Wurfeln	Hot Dice erklart
7.13	Einsteigsbedingung-Hinweis	Fehlermeldung wenn Bank < 1000 beim ersten Mal	Nutzer versteht Regel
7.14	KI-Zug-Anzeige	Waehrend KI spielt: Fortschrittsanzeige + was die KI gemacht hat	KI-Zug nachvollziehbar
7.15	Gewinner-Dialog	Modal mit Podium, alle Punkte, Button fuer neues Spiel	Spielende klar

Phase 8 – Tests & Deployment (6 Stunden)

Ziel: App laeuft stabil auf echtem Geraet, Backend auf Produktivserver validiert.

#	Was	Wie	Ergebnis
8.1	Backend-Deployment final	FileZilla, alle Dateien auf kronisoft.net/projekte/10k/	Backend erreichbar
8.2	Test: Einzelspieler + KI	Raum erstellen, 1 KI, Spiel	KI wurfelt, Runde wechselt

#	Was	Wie	Ergebnis
		starten, durchspielen	
8.3	Test: Bust	Solange würfeln bis Bust auftritt	0 Punkte, naechster dran
8.4	Test: Hot Dice	Alle 5 Würfel behalten, neu würfeln	Neue Runde startet
8.5	Test: Einsteigsbedingung	Versuchen mit < 1000 zu banken	Fehlermeldung erscheint
8.6	Test: Multiplayer 2 Geraete	2 Geraete, 1 Raum, Polling synchron	Zustand identisch
8.7	Test: App-Neustart mid-game	App schliessen, wieder oeffnen	Token geladen, direkt im Spiel
8.8	Test auf SHIFT6mq	APK auf Geraet installieren, komplettes Spiel	Stabil auf Android 13
8.9	Bugfixes	Gefundene Fehler beheben, Commits pro Fix	Stabile Version

Phase 9 – Dokumentation (4 Stunden)

Ziel: Vollstaendige IHK-konforme Projektdokumentation fuer den Abschlussbericht.

#	Was	Wie	Ergebnis
9.1	ERM-Diagramm erstellen	Draw.io oder LibreOffice Draw, 3 Tabellen mit Beziehungen	ERM-Grafik als PNG
9.2	MVVM-Architekturdiagramm	App-Schichten: View, ViewModel, Repository, API	Architektur-Grafik
9.3	API-Referenztable	Alle 8 Endpunkte: URL, Methode, Request, Response	Tabelle im Bericht
9.4	Screenflow-Skizze	Lobby > Warteraum > Spielfeld > Gewinner	Ablauf-Diagramm
9.5	Testprotokoll ausfuellen	Alle Testfaelle aus Phase 8 dokumentieren	Testprotokoll fertig
9.6	Abschlussbericht schreiben	IHK-Phasenstruktur: Analyse, Entwurf, Impl., Test, Doku	Bericht als plain text